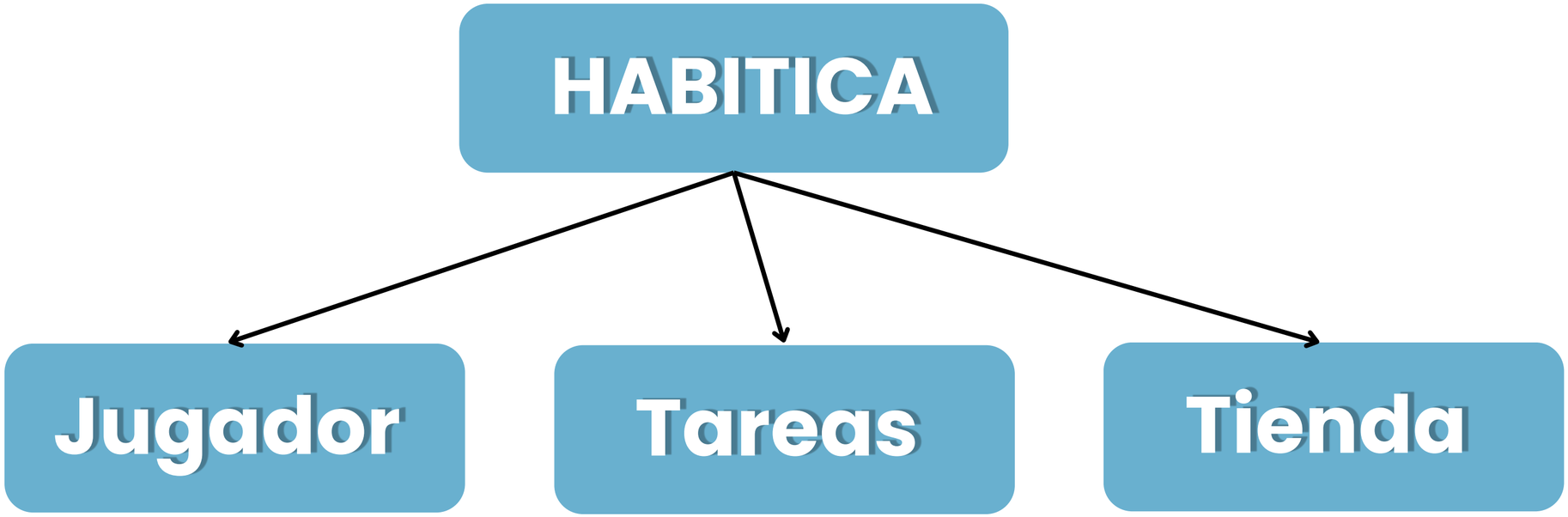
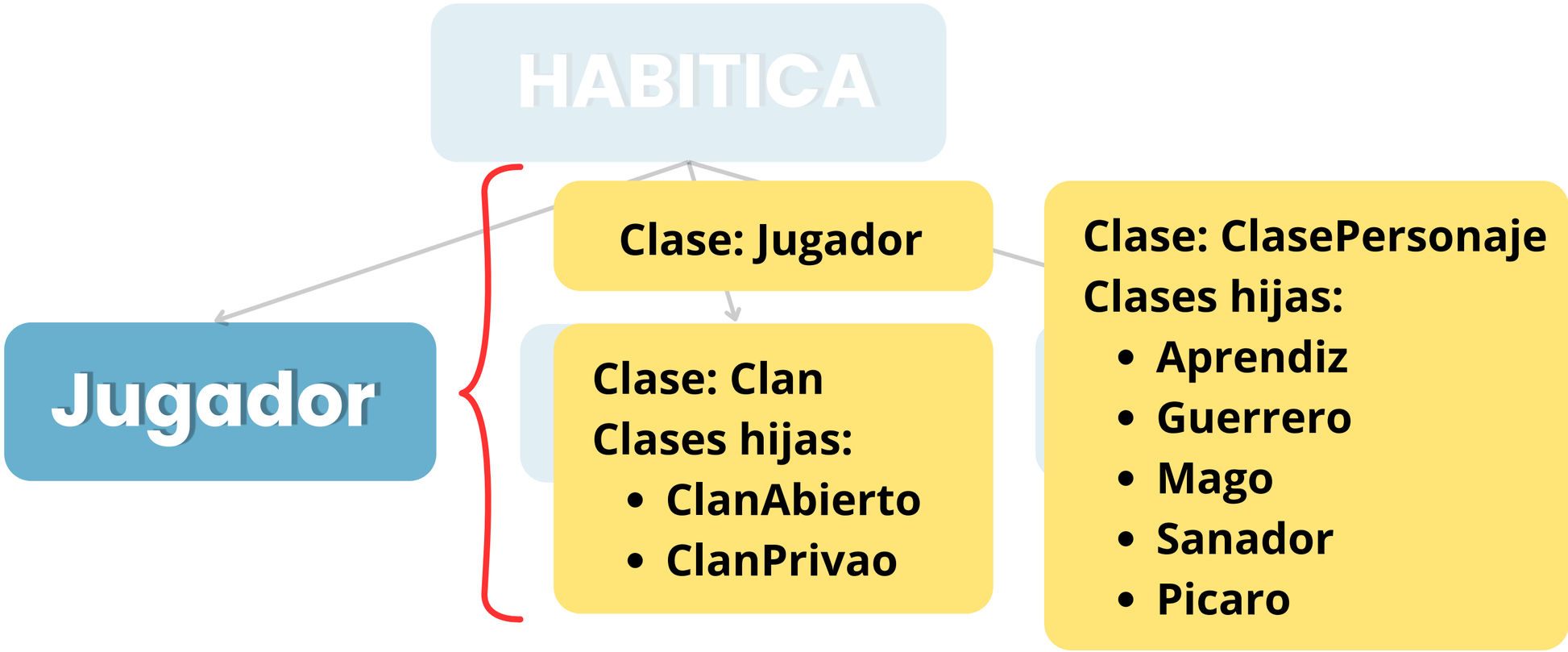
Simulando

HABITICA

Arquitectura



Arquitectura



ClaseJugador

```python
class Jugador:
    HP_MAXIMA = 50

    def __init__(self, nombre, nivel=1, clase=None):
        self.nombre = nombre
        self.nivel = nivel
        self.experiencia = 0
        self.oro = 0
        self.hp_maxima = self.HP_MAXIMA
        self.hp = self.hp_maxima
        self.clase = clase if clase is not None else Aprendiz()
        self.clan = None
        self.inventario = []

    # Experiencia necesaria para subir de nivel
    def exp_para_subir(self):
        return 50 * self.nivel
```

ClaseJugador

```python
# Gana experiencia y sube de nivel automáticamente
    def ganar_experiencia(self, cantidad):
        if cantidad > 0:
            self.experiencia += cantidad

            while self.experiencia >= self.exp_para_subir():
                self.experiencia -= self.exp_para_subir()
                self._subir_nivel()

    # Aumenta el nivel y recupera vida
    def _subir_nivel(self):
        self.nivel += 1
        self.curar(
            self.clase.curacion_al_subir_nivel(self.hp_maxima)
        )

    # Recupera vida sin superar el máximo
    def curar(self, cantidad):
        if cantidad > 0:
            self.hp = min(self.hp_maxima, self.hp + cantidad)
```

ClaseJugador

```python
class Clan:
    def __init__(self, nombre, capacidad=10):
        self.nombre = nombre
        self.capacidad = capacidad
        self.miembros = []

    # Comprueba si hay espacio disponible
    def tiene_espacio(self):
        return len(self.miembros) < self.capacidad

    # Agrega un jugador si cumple las condiciones
    def agregar_miembro(self, jugador):
        if (not self.tiene_espacio()
                or jugador in self.miembros
                or jugador.clan is not None):
            return False

        self.miembros.append(jugador)
        jugador.clan = self
        return True

    # Muestra los miembros del clan
    def mostrar_miembros(self):
        print(f"Clan: {self.nombre}")
        for jugador in self.miembros:
            print(
                f"- {jugador.nombre} "
                f"(Nivel {jugador.nivel}, {jugador.clase.nombre})"
            )
```

ClaseJugador

```python
class ClanAbierto(Clan):
    # Permite ingresar mientras haya espacio
    def unirse(self, jugador):
        return self.agregar_miembro(jugador)

class ClanPrivado(Clan):
    def __init__(self, nombre, capacidad=10):
        super().__init__(nombre, capacidad)
        self.invitados = []

    # Invita a un jugador
    def invitar(self, jugador):
        if (jugador not in self.miembros
                and jugador not in self.invitados):
            self.invitados.append(jugador)

    # Solo permite ingresar con invitación
    def agregar_miembro(self, jugador):
        return self.aceptar_invitacion(jugador)

    # Acepta la invitación e incorpora al jugador
    def aceptar_invitacion(self, jugador):
        if jugador not in self.invitados:
            return False

        if super().agregar_miembro(jugador):
            self.invitados.remove(jugador)
            return True

        return False
```

ClasePersonaje

Atributo	Descripción	Valor por defecto
`nombre`	Nombre de la clase	–
`descripcion`	Breve descripción de la clase	–
`mult_exp`	Multiplicador de experiencia	1.0
`mult_oro`	Multiplicador de oro	1.0
`reduccion_dano`	Fracción del daño que se evita	0.0
`curacion_nivel`	Fracción de vida que se restaura al subir de nivel	0.5

ClasePersonaje

```python
class ClasePersonaje:
    def __init__(self, nombre, descripcion, mult_exp=1.0, mult_oro=1.0,
                 reduccion_dano=0.0, curacion_nivel=0.5):
        self.__nombre = nombre
        self.__descripcion = descripcion
        self.__mult_exp = mult_exp
        self.__mult_oro = mult_oro
        self.__reduccion_dano = reduccion_dano
        self.__curacion_nivel = curacion_nivel
```

```python
    @property
    def nombre(self):
        return self.__nombre

    @property
    def descripcion(self):
        return self.__descripcion

    @property
    def mult_exp(self):
        return self.__mult_exp

    @property
    def mult_oro(self):
        return self.__mult_oro

    @property
    def reduccion_dano(self):
        return self.__reduccion_dano

    @property
    def curacion_nivel(self):
        return self.__curacion_nivel
```

ClasePersonaje



```
def _redondear(valor):
    return int(valor +
0.5)
```

ClasePersonaje

```python
    def __str__(self):
        return f"{self.__nombre}: {self.__descripcion}"

    def modificar_exp(self, base):
        return _redondear(base * self.__mult_exp)

    def modificar_oro(self, base):
        return _redondear(base * self.__mult_oro)

    def modificar_dano(self, base):
        # Devuelve el daño final que recibe el jugador
        if base <= 0:                # sin daño (o valor inválido)
            return 0
        # (1 - reduccion_dano) es la fracción de daño que recibe el
jugador # max(1, ...) asegura que, si hubo daño, se reciba al menos 1
        return max(1, _redondear(base * (1 - self.__reduccion_dano)))

    def curacion_al_subir_nivel(self, hp_maxima):
        return _redondear(hp_maxima * self.__curacion_nivel)
```

ClasePersonaje

```python
class Aprendiz(ClasePersonaje):
    def __init__(self):
        super().__init__("Aprendiz", "Clase inicial, sin bonificaciones")

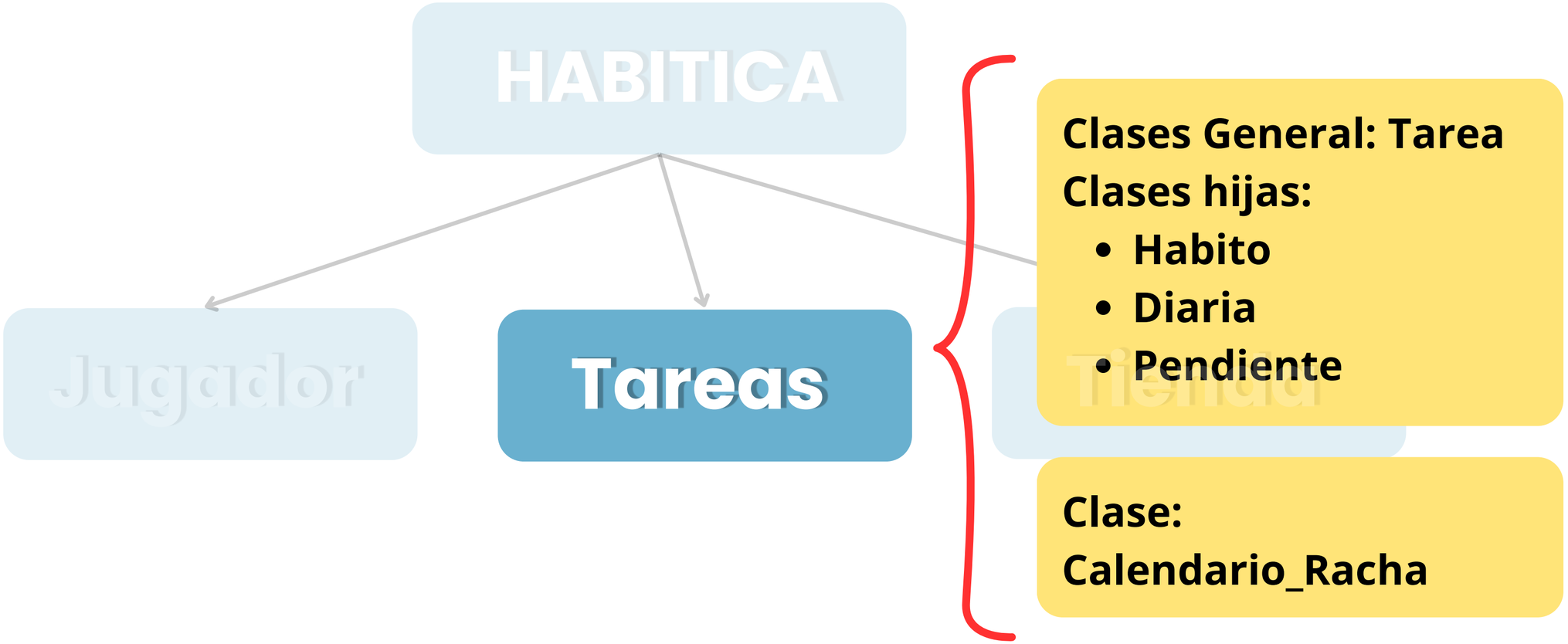

class Guerrero(ClasePersonaje):
    def __init__(self):
        super().__init__("Guerrero", "Muy resistente contra el daño", reduccion_dano=0.30)


class Mago(ClasePersonaje):
    def __init__(self):
        super().__init__("Mago", "Sube de nivel más rápido", mult_exp=1.25)


class Sanador(ClasePersonaje):
    def __init__(self):
        super().__init__("Sanador", "Restaura su salud al subir de nivel",
curacion_nivel=1.0)

class Picaro(ClasePersonaje):
    def __init__(self):
        super().__init__("Picaro", "Obtiene oro más rápido", mult_oro=1.25)
```

Arquitectura



Tarea

```python
# Importamos paquetes necesarios
from datetime import datetime
import calendar
import re

# Incluimos una variable FORMATO_FECHA auxiliar para el segundo argumento de strptime
# Incluimos una función hoy() para conocer la fecha actual del sistema
FORMATO_FECHA = "%Y-%m-%d"
def hoy():
    return datetime.now().date()
```

Tarea

Atributo	Descripción	Valor por defecto
`titulo`	Nombre de la Tarea	–
`dificultad`	Nivel de dificultad de la Tarea	-
`descripcion`	Detalles adicionales sobre la Tarea	None
`historial`	Lista de fechas en las que la Tarea fue completada	[]
`fecha_creacion`	Fecha en la que se creo un objeto de la clase Tarea	Fecha actual del sistema
`RECOMPENSAS`	Fracción de vida que se restaura al subir de nivel	Diccionario con las recompensas según la dificultad

```python
class Tarea():
    # Se define una variable diccionario RECOMPENSAS para cada tipo de dificultad.
    RECOMPENSAS = {"Facil": {"experiencia":6, "oro": 1, "vida":0},
                   "Intermedio": {"experiencia":8, "oro":1, "vida":1},
                   "Dificil": {"experiencia":12, "oro": 3, "vida":4}}

    def __init__(self, titulo:str, dificultad:str, descripcion:str = None):
        # Se valida que la dificultad sea una válida.
        if dificultad in self.RECOMPENSAS.keys():
            self.titulo = titulo
            self.dificultad = dificultad
            self.descripcion = descripcion
            self.historial = []
            self.fecha_creacion = str(hoy())

        else:
            raise ValueError(f"Se añadio una dificultad no válida.\n Debe ser una entre
                {list(self.RECOMPENSAS.keys())}")
```

Tarea

```python
    def editar_tarea(self, nuevo_titulo = None, nueva_dificultad = None, nueva_descripcion = None):
        # Cada sentencia if verifica si se modifico un atributo
        if nuevo_titulo is not None:
            self.titulo = nuevo_titulo
        if nueva_dificultad is not None:
            # Verifica que la nueva dificultad sea una válida
            if nueva_dificultad not in self.RECOMPENSAS.keys():
                raise ValueError(f"Se añadio una dificultad no válida.\n Debe ser una entre
                    {list(self.RECOMPENSAS.keys())}")
            self.dificultad = nueva_dificultad
        if nueva_descripcion is not None:
            self.descripcion = nueva_descripcion
```

Tarea

```python
    def calcular_recompensa(self):
        # Devuelve la recompensa apropiada según la dificultad de la Tarea
        return dict(self.RECOMPENSAS[self.dificultad])

    def completar(self):
        hoy1 = str(hoy())
        # Agrega la fecha de hoy en el historial de completaciones en caso no haya sido agregada
            anteriormente
        if hoy1 not in self.historial:
            self.historial.append(hoy1)
            return self.calcular_recompensa()
        return None
```

Habito

```python
class Habito(Tarea):
    # Se define una variable TIPOS_HABITO para cada tipo de habito
    TIPOS_HABITO = ["Positivo", "Negativo", "Ambos"]
    def __init__(self, titulo:str, dificultad:str, descripcion:str = None, tipo_habito = "Ambos"):
        # Verifica que el tipo de habito agregado sea uno válido
        if tipo_habito not in self.TIPOS_HABITO:
            raise ValueError(f"Se añadio un tipo de hábito no válido.\n Debe ser uno entre
                {self.TIPOS_HABITO}")
        super().__init__(titulo, dificultad, descripcion)
        self.tipo_habito = tipo_habito
        self.contador_positivo = 0
        self.historial_positivo = []
        self.contador_negativo = 0
        self.historial_negativo = []
```

Habito

```python
    @staticmethod
    def _calcular_racha(historial, contador):
        # Si no existe un historial de completaciones, la racha se vuelve 1
        if not historial:
            return 1
        ultima = datetime.strptime(historial[-1], FORMATO_FECHA).date()
        dias = (hoy()-ultima).days
        # Si la racha aumento hoy, se mantiene como estaba
        if dias == 0:
            return contador
        # Si la racha aumento ayer, se aumenta en 1 hoy
        if dias == 1:
            return contador + 1
        # Si la racha no se aumento ayer, se reinicia a 1
        return 1
```

Habito

```
    def calcular_recompensa(self, forma):
        # Verifica que se solicite la recompensa positiva o negativa de un Habito
        if forma not in ["Positivo", "Negativo"]:
            raise ValueError("La forma debe ser 'Positivo' o 'Negativo'.")
        recompensa = super().calcular_recompensa()
        # Modifica la recompensa original si el habito es Negaitvo
        if forma == "Negativo":
            recompensa["experiencia"] = recompensa["experiencia"]//2
            recompensa["oro"] = 0
            recompensa["vida"] = -1 * recompensa["vida"] - 2
        return recompensa

 def marcar_positivo(self):
        # Verifica que el tipo de Habito sea 'Positivo' o 'Ambos'
        if self.tipo_habito in ["Positivo", "Ambos"]:
            self.contador_positivo = self._calcular_racha(self.historial_positivo,
                self.contador_positivo)
            hoy1 = str(hoy())
            self.historial_positivo.append(hoy1)
            self.historial.append(hoy1)
            # Devuelve una recompensa de Habito positiva
            return self.calcular_recompensa("Positivo")
        else:
            raise ValueError("El hábito no se completo. El Hábito no es del tipo <Hábito Positivo>.")
```

Habito

```
    def marcar_negativo(self):
        # Verifica que el tipo de Habito sea 'Negativo' o 'Ambos'
        if self.tipo_habito in ["Negativo", "Ambos"]:
            self.contador_negativo = self._calcular_racha(self.historial_negativo,
                self.contador_negativo)
            hoy1 = str(hoy())
            self.historial_negativo.append(hoy1)
            self.historial.append(hoy1)
            # Devuelve una recompensa de Habito negativa
            return self.calcular_recompensa("Negativo")
        else:
            raise ValueError("El hábito no se completo. El Hábito no es del tipo <Hábito Negativo>.")

    def completar(self):
        # Cada sentencia if verifica el tipo de Habito y devuelve una recompensa según corresponda
        if self.tipo_habito == "Positivo":
            return self.marcar_positivo()
        elif self.tipo_habito == "Negativo":
            return self.marcar_negativo()
        elif self.tipo_habito == "Ambos":
            return [self.marcar_positivo(), self.marcar_negativo()]
```

Diaria

```python
class Diaria(Tarea):
    def __init__(self, titulo:str, dificultad:str, descripcion:str = None):
        super().__init__(titulo, dificultad, descripcion)
        self.racha = 0
        self.mejor_racha = 0
        self.fecha_inicio_racha = None

    def completar(self):
        # Aumenta la racha en uno
        self.racha += 1
        # Si no existe historial previo, establece la fecha de inicio de racha en hoy
        if len(self.historial) == 0:
            self.fecha_inicio_racha = str(hoy())
        # Si existe historial previo, verifica si la racha sigue vigente
        if len(self.historial) >= 2:
            if (datetime.strptime(self.historial[-1], FORMATO_FECHA) - datetime.strptime(self.historial[-2], FORMATO_FECHA)).days > 1:
                self.fecha_inicio_racha = str(hoy())
                self.racha = 1
        return super().completar()

    def fallar(self):
        # Verifica si la racha actual es mejor que la mejor racha
        if self.racha >= self.mejor_racha:
            self.mejor_racha = self.racha
        self.racha = 0
        recompensa = self.calcular_recompensa()
        recompensa["experiencia"] = 0
        recompensa["oro"] = 0
        recompensa["vida"] = -10
        return recompensa
```

Pendiente

```python
class Pendiente(Tarea):
    def __init__(self, titulo:str, dificultad:str, fecha_limite:str, descripcion:str = None):
        max = datetime.strptime(fecha_limite, FORMATO_FECHA).date()
        super().__init__(titulo, dificultad, descripcion)
        # Verifica si la fecha máxima para completar el Pendiente aún no ocurre
        if (max - hoy()).days >= 0:
            self.fecha_limite = fecha_limite
        else:
            raise ValueError("La fecha limite debe ser una igual o posterior a la de hoy.")
        self.completada = False
        self.vencida = False

    def esta_vencida(self):
        self.vencida = True

    def calcular_recompensa(self):
        recompensa = super().calcular_recompensa()
        # Verifica si el Pendiente esta vencido
        if self.vencida:
            recompensa["experiencia"] -= 4
            recompensa["oro"] -= 1
            recompensa["vida"] = 0
        return recompensa
```

Calendario_Racha

Atributo	Descripción	Valor por defecto
`tarea`	Tarea de la cual se busca mostrar su calendario	–

```python
class Calendario_Racha():
    def __init__(self, tarea):
        # Verifica que el parametro clase tarea sea hijo de Tarea
        if not issubclass(type(tarea), Tarea):
            raise ValueError("El argumento <tarea> debe ser de la clase Tarea")
        self.tarea = tarea
```

```
    January 2026
Mo Tu We Th Fr Sa Su
                1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31
```

```
   February 2026
Mo Tu We Th Fr Sa Su
                   1
 2  3  4  5  6  7  8
 9 10 11 12 13 14 15
16 17 18 19 20 21 22
23 24 25 26 27 28
```

```
     March 2026
Mo Tu We Th Fr Sa Su
                   1
 2  3  4  5  6  7  8
 9 10 11 12 13 14 15
16 17 18 19 20 21 22
23 24 25 26 27 28 29
30 31
```

Calendario_Racha

```
    def mostrar(self, tipo_historial = None):
        # Verifica si se relleno el parametro tipo_historial para objetos que no son instancias de Habito.
        if not isinstance(self.tarea, Habito) and tipo_historial is not None:
            raise ValueError(f"El parametro <tipo_historial> solo debe ser ingresado para instancias de Habito.")
        # Verifica si no se relleno el parametro tipo_historial para objetos instancias de Habito.
        elif isinstance(self.tarea, Habito) and tipo_historial is None:
            raise ValueError(f"El parametro <tipo_historial> debe completarse para instancias de Habito.")
        tipos_historial = ["historial_positivo", "historial_negativo", "historial_completo"]
        # Verifica si el parametro tipo_historial no se completo
        if tipo_historial is None:
            tipo_historial = "historial_completo"
        # Verifica si el parametro tipo_historial es un tipo de historial válido
        if tipo_historial not in tipos_historial:
            raise ValueError(f"El tipo de historial ingresado no es un tipo válido para la clase Habito. El
                <tipo_historial> debe ser uno entre {tipos_historial}")

        if tipo_historial == "historial_completo":
            fechas_str = self.tarea.historial
        elif tipo_historial == "historial_positivo":
            fechas_str = self.tarea.historial_positivo
        elif tipo_historial == "historial_negativo":
            fechas_str = self.tarea.historial_negativo

        # Verifica si existe historial de completaciones
        if len(fechas_str) == 0:
            anio = int(self.inicio[:4])
            mes = int(self.inicio[5:7])
            print(calendar.month(anio, mes))
            return
```

Calendario_Racha

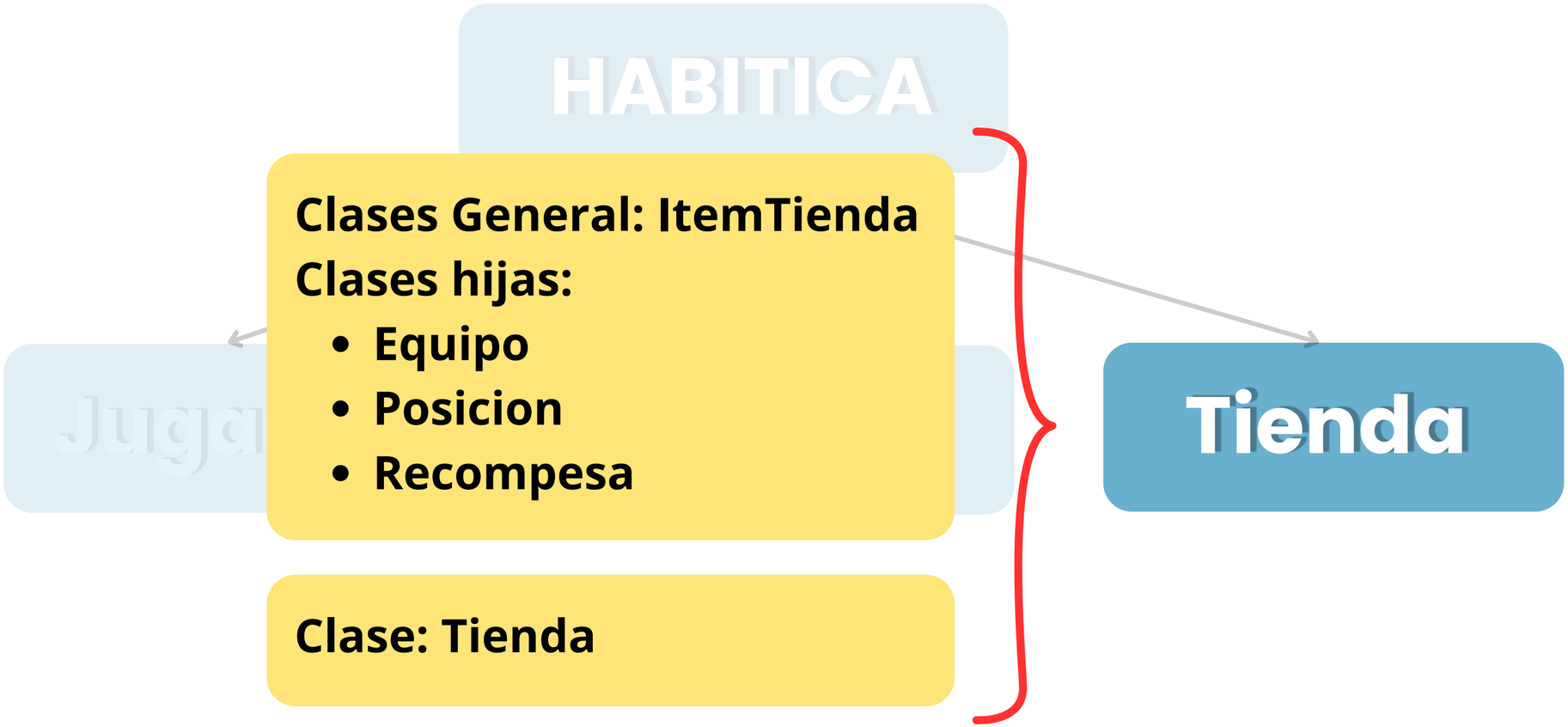
```
        dict_fechas = {}
        for fecha in fechas_str:
            # Añade a dict_fechas cada fecha no repetida del historial de completaciones
            fecha_dt = datetime.strptime(fecha, FORMATO_FECHA)
            key_mes = (fecha_dt.year, fecha_dt.month)
            if key_mes not in dict_fechas.keys():
                dict_fechas[key_mes] = []
            dict_fechas[key_mes].append(str(fecha_dt.day))

        ults_tres_meses = sorted(dict_fechas.keys())[-3:]

        # Selecciona cada combinacion de año, mes
        for anio_mes in ults_tres_meses:
            anio, mes = anio_mes
            dias = dict_fechas[anio_mes]
            lineas = calendar.month(anio, mes).split("\n")

            # Colorea cada fecha en la que se completo la Tarea
            for i in range(2, len(lineas)):
                for dia in dias:
                    patron = rf"\b{dia}\b"
                    reemplazo = f"\033[91m{dia}\033[0m"
                    lineas[i] = re.sub(patron, reemplazo, lineas[i])
            print("\n".join(lineas))
```

Arquitectura



Clase Base: ItemTienda

Atributo	Descripción	Valor por defecto
`_contador_codigo`	Generador autoincremental de IDs.	1
`codigo`	Identificador único del ítem.	Automático
`nombre`	Nombre visible en la tienda.	-
`precio`	Costo de compra en oro.	-

Implementación ItemTienda

```python
class ItemTienda:
    _contador_codigo = 1

    def __init__(self, nombre, precio, nivel_requerido, descripcion):
        if precio < 0:
            raise ValueError("El precio no puede ser negativo.")
        if nivel_requerido < 1:
            raise ValueError("El nivel requerido debe ser al menos 1.")
        self.codigo = ItemTienda._contador_codigo
        ItemTienda._contador_codigo += 1
        self.nombre = nombre
        self.precio = precio
        self.nivel_requerido = nivel_requerido
        self.descripcion = descripcion

    def aplicar(self, jugador):
        raise NotImplementedError("Cada tipo de ítem define su propio efecto.")
```

Clases hijas (Polimorfismo)

Equipo

Atributo `bonificacion`. Aumenta la vida máxima y se guarda en el inventario.

Poción

Atributo `hp_restaurada`. Cura al jugador respetando su límite de vida máxima.

Recompensa

Premio personalizado canjeable creado por el propio usuario.

Implementación Clases Hijas

```python
class Equipo(ItemTienda):
    def __init__(self, nombre, precio, nivel_requerido, descripcion, bonificacion):
        if bonificacion <= 0:
            raise ValueError("La bonificación debe ser positiva.")
        super().__init__(nombre, precio, nivel_requerido, descripcion)
        self.bonificacion = bonificacion    # puntos que aumentan la hp_maxima del jugador

    def aplicar(self, jugador):
        # Se guarda en el inventario y mejora la vida máxima (sin repetir el mismo equipo)
        if self in jugador.inventario:
            print(f"Ya tienes equipado: {self.nombre}.")
            return False
        jugador.inventario.append(self)
        jugador.hp_maxima += self.bonificacion
        print(f"Has equipado: {self.nombre}. Bonificación: +{self.bonificacion} HP máxima")
        return True
```

Implementación Clases Hijas

```python
class Pocion(ItemTienda):
    def __init__(self, nombre, precio, nivel_requerido, descripcion, hp_restaurada):
        if hp_restaurada <= 0:
            raise ValueError("La vida restaurada debe ser positiva.")
        super().__init__(nombre, precio, nivel_requerido, descripcion)
        self.hp_restaurada = hp_restaurada

    def aplicar(self, jugador):
        # Cura al jugador respetando su hp_maxima (usa Jugador.curar)
        if jugador.hp >= jugador.hp_maxima:
            print("Tu salud ya está al máximo; no se usó la poción.")
            return False
        hp_antes = jugador.hp
        jugador.curar(self.hp_restaurada)
        print(f"Te has curado {jugador.hp - hp_antes} HP. Salud actual:
{jugador.hp}/{jugador.hp_maxima}")
        return True
```

Implementación Clases Hijas

```python
class Recompensa(ItemTienda):
    def __init__(self, nombre, precio, nivel_requerido, descripcion, creada_por_usuario):
        super().__init__(nombre, precio, nivel_requerido, descripcion)
        self.creada_por_usuario = creada_por_usuario

    def aplicar(self, jugador):
        # Solo se registra el premio real
        print(f"Recompensa canjeada: {self.nombre}. ¡Disfruta tu premio!")
        return True
```

Clase Gestora: Tienda

```python
class Tienda:
    def __init__(self):
        self.items = []    # Lista de objetos ItemTienda

    def agregar_item(self, item):
        if not isinstance(item, ItemTienda):
            raise TypeError("Solo se pueden agregar objetos de tipo ItemTienda.")
        if item not in self.items:
            self.items.append(item)

    def mostrar_tabla(self):
        print("\n--- OBJETOS DE LA TIENDA ---")
        if not self.items:
            print("La tienda está vacía.")
            return
        for item in self.items:
            print(f"[{item.codigo}] {item.nombre} | Nivel req: {item.nivel_requerido} | Precio:
{item.precio} oro")
            print(f"    Desc: {item.descripcion}")

    def buscar_por_codigo(self, codigo):
        for item in self.items:
            if item.codigo == codigo:
                return item
        return None
```

Clase Gestora: Tienda

```python
    def comprar(self, jugador, codigo):
        item = self.buscar_por_codigo(codigo)
        if item is None:
            print(f"No existe un ítem con código {codigo}.")
            return False
        if jugador.nivel < item.nivel_requerido:
            print(f"Necesitas nivel {item.nivel_requerido} para comprar {item.nombre}.")
            return False
        if jugador.oro < item.precio:
            print(f"Oro insuficiente para {item.nombre}: tienes {jugador.oro} y cuesta {item.precio}.")
            return False
        if not item.aplicar(jugador):
            return False
        jugador.oro -= item.precio
        print(f"Compra realizada: {item.nombre} (-{item.precio} oro). Oro restante: {jugador.oro}")
        return True
```